

Voronoi Diagrams, Optimal Transport and Fluids Report

Minh Hoang Duong

June 16, 2026

1 Introduction

In this report, we will demonstrate the results of a small geometry processing project written from scratch in C++. Notably, we will go over computing Voronoi diagrams, extending them to Laguerre cells with semi-discrete optimal transport, and simulating a 2D fluid with partial optimal transport. All three labs build on top of each other, and they all use the same two tools: a polygon clipping routine (Sutherland–Hodgman) and the L-BFGS optimizer. The ultimate end goal is making a pool of water fluid fall inside a box. All experiments were run on my personal laptop, 16 cores, with the heavy loops parallelized using OpenMP.

2 Voronoi diagrams (Lab 1)

First and foremost, the basic building block of the project is the Voronoi cell. Given a set of sites in the unit square, the Voronoi cell of a site is the region of points that are closer to that site than to any other site, where each cell are calculated separately and in parallel. For each site, we start with the full unit square as a polygon, and for every other site we clip the cell by the bisector of the segment between the two sites, keeping only the half closer to our site. The clipping is done with the Sutherland–Hodgman algorithm, which walks along the edges of the polygon and adds an intersection point whenever an edge crosses the cutting line.

In this first lab, we tested with 100 random sites, which is enough to check that the diagram tiles the whole square without holes or overlaps. One thing to be careful about: the same compute function is later reused for the fluid, where it also clips each cell by a disk. Total time to compute is under 5ms for 100 sites.

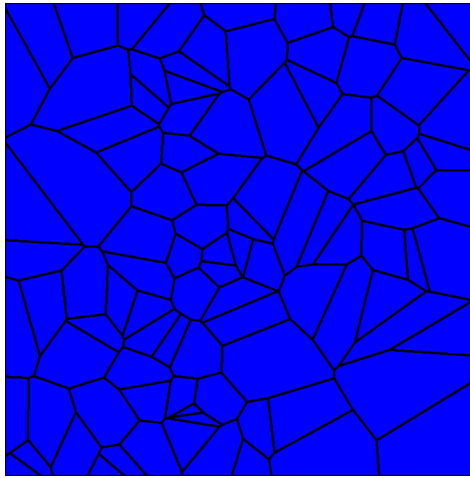


Figure 1: Voronoi diagram of 100 random sites

3 Semi-discrete optimal transport (Lab 2)

To control the area of each cell, we extend the Voronoi cell into a Laguerre cell/power cell. Each site now has a weight, and the comparison used during clipping is shifted by these weights instead of just distance comparison. Increasing the weight of a site makes its cell grow and pushes against its neighbors.

To assign these weights properly, we use semi-discrete optimal transport, which chooses the weights so that every cell reaches a target area. This is done by maximizing a function whose gradient, for each site, is the difference between the target area and the current cell area. So if a cell is too small the optimizer raises its weight, and if it is too big it lowers it, until all the areas match. We pass this function and its gradient to L-BFGS, and at every iteration we recompute the whole Laguerre diagram with the new weights. The squared-distance term in the function is computed exactly by splitting each cell into triangles. For this experiment we used a uniform target, so every cell is convergent to the same area.

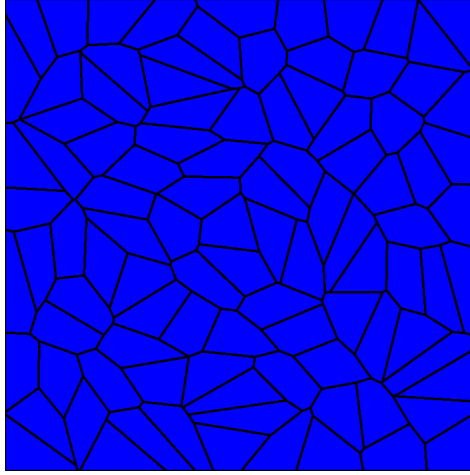


Figure 2: Laguerre diagram after optimal transport, with a uniform target area

The optimizer converges in around 140 iterations, at which point the gradient norm is very small and every cell area matches its target. The result in Figure 2 is the Voronoi diagram from before but with the cell sizes evened out, which is what we expect with a uniform target. The optimization is still very fast, since each iteration only recomputes 100 cells.

4 Partial optimal transport and fluid simulation (Lab 3)

Finally, we reuse the same tools to simulate fluid. The fluid is similar to a set of particles (points), and each particle owns its own Laguerre cell. To model the boundary between the fluid and the surrounding air, we use partial optimal transport. For this, we added a single extra air weight, and clipping each fluid cell not only by its neighbours but also by a small disk around its particle. The disk is approximated by a 32-sided polygon and clipped with the same routine as before. The transport step keeps all the fluid cells together taking up a fixed fraction of the box, leaving the rest to the air.

Each time step has two parts. First, we solve the optimal transport to get the cells for the current particle positions. Then, for every particle, we apply forces: gravity, and a spring force pulling the particle towards its own cell. We then integrate the forces into velocities and the velocities into positions, and clamp any particle that tries to leave the box. The fluid starts as a disk of 1000 particles in the upper part of the box, which then falls under gravity and spreads out into a pool at the bottom, as shown in Figure 3.

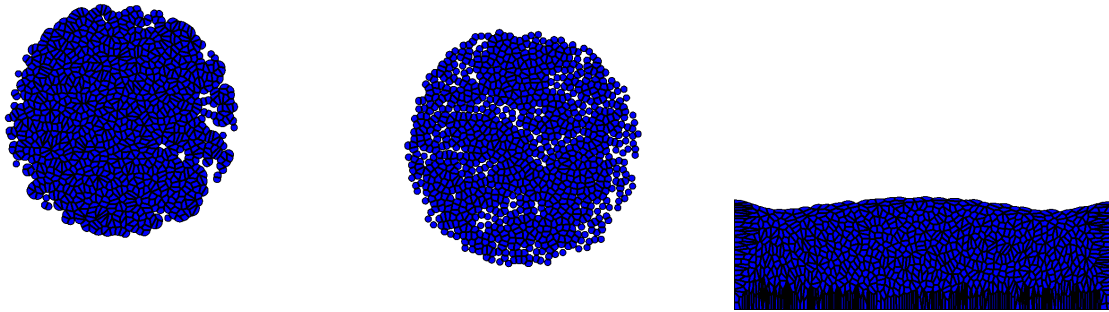


Figure 3: Fluid simulation: the initial ball of fluid (left), spreading as it falls (middle), and the settled at the bottom of the box (right)

I rendered all 1000 frames and assembled them into a single video using `ffmpeg`, included in the repository as `output.mp4`. The full video can be seen here: [GitHub \(output.mp4\)](#).

4.1 Optimizations

The fluid is by far the heaviest part, since it solves a full optimal transport at every frame. Originally it was very slow: the first 300 frames alone took about 20 minutes (Table 1). Two changes fixed most of it. First, we loosened the L-BFGS tolerance (epsilon) to $1e-3$: the previous frame’s weights are already a good guess, so there is no need to fully converge, and the calm frames drop from about 34 iterations to 3. Second, instead of clipping each cell against all 999 other particles, we keep only its k nearest neighbours (with $k = 80$), since a cell only really borders the particles close to it. Together the two changes take those 300 frames from 20 minutes to about 70 seconds.

Version (first 300 frames)	total time	per frame
Original (all neighbours, full convergence)	~ 20 min	3.94s
Optimized (k nearest neighbours, loose tolerance)	70s	0.23s

Table 1: Effect of the two optimizations, on the first 300 frames.

5 Conclusion

Starting from a single polygon clipping routine, we built a Voronoi diagram, turned it into a Laguerre diagram solved by optimal transport, and reused the same ideas with partial optimal transport to simulate a fluid falling and settling under gravity. The code and this report are available on [GitHub](#).